

L3 Patterns-in-Code Document — HELEP

Part A — Pre-implemented patterns

A.1 Choreographed Saga

Where:

- `sos-service/app/main.py:85` — `trigger()` calls `await publish("sos.triggered", ...)`
- `dispatch-service/app/main.py` — consumes `sos.triggered`, assigns responder, publishes `responder.assigned`
- `notification-service/app/main.py` — consumes `responder.assigned`, logs SMS, publishes `notification.sent`

Compensation step: When citizen calls `POST /sos/{id}/cancel`, `sos-service/app/main.py:109` publishes `sos.cancelled`. `dispatch-service/app/main.py` consumes it and executes rollback: sets `responders.busy=0` and marks assignment row as `RELEASED`.

Rollback trigger event: `sos.cancelled`

Why choreography not orchestration: No central coordinator exists. Each service reacts to events independently — if `dispatch-service` crashes, it replays from Kafka on restart without losing saga state.

A.2 Pub/Sub via Apache Kafka

Where: `app/events.py` in every service — `AIOKafkaProducer` (publish) and `AIOKafkaConsumer` (subscribe).

Consumer group semantics: Each service has its own `group_id` (e.g. `dispatch-service`, `notification-service`). This means:

- All 5 services receive the same `sos.triggered` event independently
- If `dispatch-service` has 3 replicas, Kafka splits partitions between them so only ONE replica processes each event — no double dispatch

At-least-once delivery: `enable_auto_commit=False` in every `consume()` call — the consumer only calls `await consumer.commit()` AFTER the handler succeeds. If the handler crashes, the offset is not committed so the event is redelivered on restart.

Partition keying: `sos-service/app/main.py:85` passes `key=iid` (`incident_id`) to every saga-critical publish. Same `incident_id` always routes to the same partition → ordering preserved → no double dispatch even with multiple replicas.

A.3 Repository

Where: `app/db.py` in every service.

- `user-service/app/db.py:13` — `conn()` context manager owns all DB connections
- `user-service/app/db.py:46` — `insert_user()` — only place that writes users table
- `user-service/app/db.py:54` — `find_by_phone()` — only place that reads users table

`main.py` never writes SQL directly — it only calls these functions.

Why: If route handlers queried SQLite directly, a schema change would require hunting through all endpoint code. Repository centralises all data access in one file — change schema in one place only.

A.4 Strategy

Where: dispatch-service/app/matching.py

- matching.py:31 — class NearestMatcher — picks smallest haversine distance
- matching.py:42 — class CredibilityWeightedMatcher — scores by credibility / (distance_km + 1)
- matching.py:56 — class RoundRobinMatcher — assigns in turn (added by student)
- matching.py:28 — def pick() — common interface all three implement

How to switch: Set environment variable MATCHER=nearest, MATCHER=credibility, or MATCHER=roundrobin. The matcher() factory function reads this and returns the correct instance. No if-chains in business logic — open/closed principle satisfied.

A.5 Outbox-lite

Where: sos-service/app/main.py:84-85

insert_incident(iid, ...) # line 84 — write to SQLite first

await publish("sos.triggered", ...) # line 85 — then publish to Kafka

Why "lite": A real Outbox pattern uses a dedicated outbox table and a separate polling process. Our version skips the polling process — if the service crashes after line 84 but before line 85, the event is lost. Acceptable for dev; production needs full transactional outbox.

A.6 Circuit Breaker

Where: services/user-service/app/events.py:59 — class CircuitBreaker

- events.py:59 — class definition
- events.py:66 — allow() — implements CLOSED/OPEN/HALF-OPEN state machine
- events.py:74 — record_success() — resets to CLOSED
- events.py:78 — record_failure() — trips to OPEN after 5 failures

Completed state machine:

- CLOSED: opened_at is None → allow() returns True
- OPEN: fails >= 5 → opened_at set → allow() returns False
- HALF-OPEN: after 10s → allow() returns True for one test request

Why it matters: Without Circuit Breaker, every publish() blocks and times out when Kafka is down, cascading failure to all HTTP handlers. Circuit Breaker fails fast after 5 failures.

B.1 Bulkhead

Pattern source: Cloud-Native catalogue (Netflix)

Where: Architecture-level — each service passes its own `group_id` to `consume()` in `app/events.py` in every service. Example: `dispatch-service` uses `group_id="dispatch-service"`, `analytics-service` uses `group_id="analytics-service"`.

Problem solved in HELEP: If `analytics-service` is slow processing events, without Bulkhead it would slow the shared consumer and delay dispatch assignments. Isolated consumer groups mean a slow analytics consumer never blocks `dispatch-service` from reading `sos.triggered`.

Trade-off: Kafka delivers each event to multiple consumer groups — higher broker load vs. better fault isolation. Worth it for ASR-1.

B.2 Idempotency Key

Pattern source: Enterprise Integration Patterns (Hohpe & Woolf)

Where: `dispatch-service/app/db.py` — assignment table uses `incident_id` as PRIMARY KEY with INSERT OR IGNORE semantics.

Problem solved in HELEP: Kafka at-least-once delivery means `sos.triggered` can be redelivered if consumer crashes before committing offset. Without idempotency, `dispatch-service` would assign two responders to the same incident. The PRIMARY KEY constraint makes the second INSERT a no-op — exactly-once assignment regardless of redeliveries.

Trade-off: Requires globally unique `incident_id` per event. Alternative pessimistic locking rejected — reduces throughput.

Part C — Anti-pattern avoided

Anti-pattern: Shared Database

HELEP explicitly avoids the Shared Database anti-pattern. Each service owns its own isolated SQLite file via `DB_PATH` env var:

- `user-service/app/db.py` — manages only users and contacts tables
- `dispatch-service/app/db.py` — manages only responders and assignments tables

No service reads or writes another service's database. No cross-service JOIN queries exist anywhere in the codebase.

Why it matters: A shared database couples services at the schema level — a migration in one service's table can break another at runtime. Isolated databases let each service evolve independently, satisfying the Portability NFR (SRS §3).